

Apache Kafka

Avro

Streaming Analytics

Retry

DLQ

Docker

Kafka Avro Order Analytics

Complete Demonstration Video Script and Evidence Guide

Student: _____

Registration number: _____

Module: Big Data Analytics

Recommended duration: 8–10 minutes

Project folder: E:\FYP\fyp_new\big data analytics

Use this as a spoken script. Text under "SAY THIS" can be read nearly word-for-word. Text under "ON SCREEN" tells you what to display.

1. Before recording

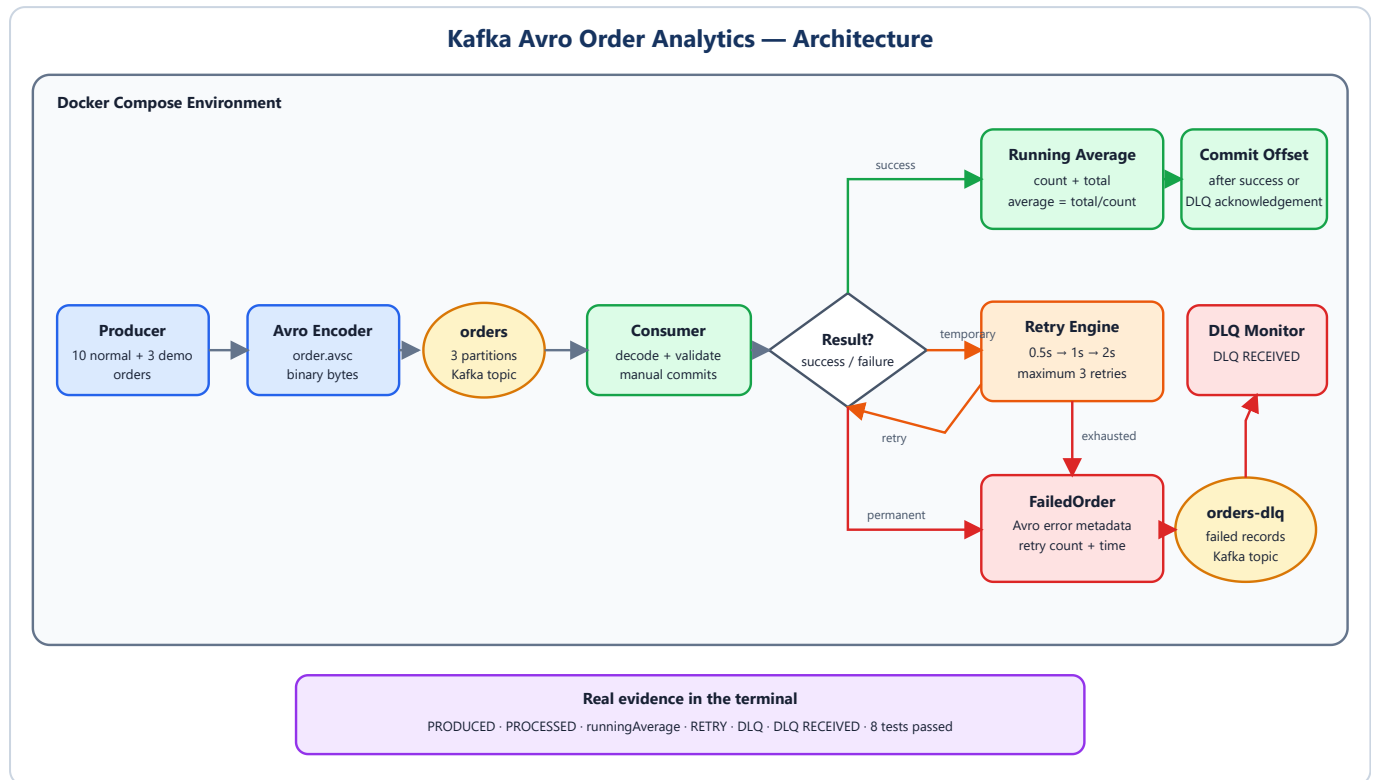
- ☐ Add your name and registration number to `REPORT.md` and this cover.
- ☐ Start Docker Desktop and wait until it says the engine is running.
- ☐ Close unrelated windows, messages, passwords and notifications.
- ☐ Open VS Code with the `big data analytics` folder visible.
- ☐ Open a PowerShell terminal and increase its font size.
- ☐ Open `kafka-order-analytics-architecture.drawio` in draw.io.
- ☐ Record at 1080p if possible and verify microphone audio first.

Important: Run `docker compose down -v` before the demonstration. This removes previous demo containers and Kafka data so the video starts from clean topics and offsets.

2. Video plan

Time	Section	Evidence or concept
0:00–0:35	Introduction	Assignment goal and implemented requirements
0:35–1:50	Architecture diagram	Producer → Avro → Kafka → consumer → average/retry/DLQ
1:50–3:35	Folder walkthrough	Schemas, codec, producer, processing, consumer, Docker, tests
3:35–6:50	Live demonstration	Produced records, aggregation, retries and DLQ evidence
6:50–7:45	Infrastructure and tests	Topics, eight tests, Git history
7:45–9:15	Theory and limitations	Partitions, offsets, delivery semantics and improvements
9:15–9:40	Conclusion	Requirement-by-requirement summary

3. Architecture diagram



SAY THIS

"This diagram shows the complete event flow. The producer creates normal and controlled failure orders. Each order is encoded into Avro binary and published to the partitioned Kafka orders topic. The consumer decodes and validates it. Successful orders update an incremental running average. Temporary failures enter bounded exponential retry. Permanent failures and exhausted retries are encoded with diagnostic metadata and written to the orders-dlq topic. The source offset is committed only after successful processing or an acknowledged DLQ write."

How to explain every architecture component

Component	What it does	Theory to mention
Producer	Creates 10 random orders and three controlled failure cases.	Producers publish events without directly depending on consumers.
Avro encoder	Uses <code>order.avsc</code> to turn a Python dictionary into compact bytes.	Schema-based serialization creates a typed producer-consumer contract.
<code>orders</code>	Stores records across three partitions using <code>orderId</code> as the key.	Partitions enable parallelism; ordering is guaranteed only inside a partition.
Consumer	Polls messages, decodes Avro and performs validation and processing.	A consumer group tracks work through partition offsets.
Running average	Updates count and total for each successful order.	Incremental stream aggregation uses $O(1)$ state per update.
Retry engine	Retries temporary failures after 0.5, 1 and 2 seconds.	Bounded exponential backoff gives dependencies time to recover without infinite retry.
DLQ	Preserves permanent and retry-exhausted messages with error metadata.	A Dead Letter Queue isolates poison messages for diagnosis or reprocessing.
Manual commit	Commits only after success or confirmed DLQ publication.	This avoids acknowledging a record before its outcome is safely handled.

4. Scene-by-scene recording script

Scene 1 — Introduction (0:00–0:35)

ON SCREEN

Show the project root in VS Code with `app`, `schemas`, `tests`, `docker-compose.yml`, `README.md` and `REPORT.md`.

SAY THIS

"Hello, this is my Big Data Analytics assignment. I implemented a real-time Kafka order-processing pipeline using Python, Apache Kafka and Avro. The project demonstrates Avro serialization, a producer and consumer, a continuously updated running price average, bounded retry logic for temporary failures, a Dead Letter Queue for unrecoverable messages, automated tests, Docker deployment and Git version control."

Scene 2 — Architecture (0:35–1:50)

ON SCREEN

Open `kafka-order-analytics-architecture.drawio`. Zoom so the complete flow is readable. Follow the arrows from left to right, then show the success, retry and DLQ branches.

SAY THIS

"The producer is decoupled from the analytics logic. It sends Avro binary events to the orders topic instead of calling the consumer directly. Kafka durably stores those events. The topic has three partitions, allowing scalable processing. The consumer belongs to a consumer group, polls records by partition and offset, deserializes the Avro payload and validates the order. A successful order updates the running count, total and average. A temporary error enters exponential retry. A permanent error or exhausted retry is written to the Dead Letter Queue. Finally, Kafka offsets are committed manually only after the record has a successful outcome."

Scene 3 — Avro schemas (1:50–2:15)

ON SCREEN

Open `schemas/order.avsc`, followed by `schemas/failed_order.avsc`.

SAY THIS

"The Order schema defines exactly three required fields: `orderId` and `product` as strings, and `price` as a float. This is the contract used by both producer and consumer. The `FailedOrder` schema extends the business data with error type, error message, retry count and UTC failure time. Therefore, normal and failed Kafka values are both Avro serialized."

Scene 4 — Avro codec and producer (2:15–2:50)

ON SCREEN

Open `app/avro_codec.py` . Highlight `schemaless_writer` and `schemaless_reader` . Then open `app/producer.py` and show `acks=all` , `enable.idempotence` , the three demo orders and `encode_avro` .

SAY THIS

"The codec parses and caches each schema, writes records into Avro binary bytes and reconstructs them during reading. The producer enables acknowledgements and idempotence to improve delivery reliability. It generates normal randomized orders plus three deterministic scenarios: one temporary failure that recovers, one permanent failure and one temporary failure that never recovers. The order ID becomes the Kafka key, while the Avro bytes become the Kafka value."

Scene 5 — Processing and aggregation (2:50–3:15)

ON SCREEN

Open `app/processing.py` . Highlight `RunningAverage` , `validation` and the three special product names.

SAY THIS

"The running aggregation keeps only a count and cumulative total. For each successful price, it calculates total divided by count. This is incremental streaming analytics, so it does not need to retain every historical order. Validation rejects missing identifiers, empty product names and non-positive or non-finite prices as permanent errors. The special product values create predictable demonstration failures."

Scene 6 — Consumer, retry and DLQ (3:15–3:35)

ON SCREEN

Open `app/consumer.py` . Highlight `enable.auto.commit=False` , the retry formula, `send_to_dlq` and the final synchronous commit.

SAY THIS

"Automatic commits are disabled so the application controls when a message is acknowledged. Temporary errors use bounded exponential backoff. Permanent errors skip retry and go directly to the DLQ. The DLQ producer flushes and verifies delivery, and only then does the consumer commit the source message offset."

5. Live demonstration

Scene 7 — Start from clean state (3:35–4:10)

ON SCREEN

Open PowerShell and run the following commands one by one.

```
cd "E:\FYP\fyp_new\big data analytics"  
docker compose down -v  
docker compose up --build -d  
docker compose ps --all
```

SAY THIS

"I am removing previous demonstration data so Kafka starts with clean topics and consumer offsets. Docker Compose now starts Kafka in KRaft mode, creates the two topics, starts the analytics and DLQ consumers, and finally runs the producer. The Kafka service should be healthy, the two consumers should remain running, and the producer should exit successfully after publishing its batch."

Scene 8 — Show all runtime evidence (4:10–6:30)

ON SCREEN

Run this command and slowly scroll through the output.

```
docker compose logs --no-color producer consumer dlq-consumer
```

Show and explain the following six categories of evidence.

1. Production evidence

```
producer | Producing 13 Avro orders to orders  
producer | PRODUCED key=... topic=orders partition=... offset=...  
producer | Producer finished
```

Say: "This proves that 13 Avro messages were acknowledged by Kafka. Every delivery report identifies the topic, selected partition and assigned offset."

2. Successful processing and real-time aggregation

```
consumer | PROCESSED orderId=... price=... count=... runningAverage=...
```

Say: "Every successful event immediately changes the count and average. The output is streaming evidence because the result is updated as records arrive rather than after a static batch query."

3. Recoverable temporary failure

```
consumer | RETRY orderId=demo-retry-success retry=1/3 delay=0.50s
consumer | RETRY orderId=demo-retry-success retry=2/3 delay=1.00s
consumer | PROCESSED orderId=demo-retry-success price=125.00 ...
```

Say: "This dependency failure is temporary. The record fails twice, waits using exponential backoff and then succeeds on its third processing attempt. Because it succeeds, it contributes to the running average and does not enter the DLQ."

4. Permanent failure

```
consumer | DLQ orderId=demo-permanent error=PermanentProcessingError retries=0
```

Say: "This is classified as permanent, so no retry is attempted. Repeating invalid business data cannot fix it, so it moves directly to the DLQ."

5. Retry exhaustion

```
consumer | RETRY orderId=demo-retry-exhausted retry=1/3 delay=0.50s
consumer | RETRY orderId=demo-retry-exhausted retry=2/3 delay=1.00s
consumer | RETRY orderId=demo-retry-exhausted retry=3/3 delay=2.00s
consumer | DLQ orderId=demo-retry-exhausted error=TemporaryProcessingError retries=3
```

Say: "This failure remains temporary in classification, but it does not recover within the configured retry budget. After three retries, the consumer prevents an infinite loop by moving it to the DLQ."

6. Independent DLQ confirmation

```
dlq-consumer | DLQ RECEIVED orderId=demo-permanent ... retries=0
dlq-consumer | DLQ RECEIVED orderId=demo-retry-exhausted ... retries=3
```

Say: "These lines come from a separate consumer of the orders-dlq topic. They prove that failed records were not only logged by the main consumer; they were actually serialized, published to Kafka and consumed from the DLQ."

If output order differs: Explain that Kafka guarantees ordering inside one partition only. The `orders` topic has three partitions, so log lines from different partitions can legitimately appear in a different order.

Scene 9 — Kafka topics (6:30–6:50)

ON SCREEN

Run:

```
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:29092 --list
```

SAY THIS

"The broker contains the normal orders topic and the separate orders-dlq topic. This confirms that the failure channel is implemented as a real Kafka topic."

6. Tests and Git evidence

Scene 10 — Automated tests (6:50–7:20)

ON SCREEN

Run:

```
docker compose run --rm --no-deps consumer python -m pytest -q
```

SAY THIS

"The eight automated tests verify the running-average calculation, Avro round-trip encoding and decoding, recovery after temporary failures, direct permanent failure handling and invalid price validation. The result should show eight passed."

Scene 11 — Git repository (7:20–7:45)

ON SCREEN

Run:

```
git log --oneline -1  
git status
```

SAY THIS

"The assignment folder is maintained as its own Git repository on the main branch. The log shows the assignment commit, while Git status demonstrates the current working-tree state. This satisfies the version-control part of the assignment."

7. Core theory to explain

Kafka topic, partition, key and offset

SAY THIS

"A Kafka topic is a named event stream. A partition is an ordered section of that stream and allows work to scale in parallel. The order ID is the message key, so messages with the same key are routed consistently. An offset is a message's position inside one partition. A consumer group coordinates which consumer owns each partition and remembers processing progress through committed offsets."

Why Kafka is appropriate for big data

SAY THIS

"Kafka is designed for continuous high-velocity event ingestion. It decouples producers and consumers, stores events durably, supports replay and scales throughput through partitions. This demonstration uses a small number of orders, but the architecture represents a scalable streaming system."

Why Avro instead of plain JSON

SAY THIS

"Avro provides compact binary encoding and explicit typed schemas. The schema prevents arbitrary message structures and forms a clear contract between producer and consumer. This project deploys the schema files with both applications. A larger system could add a Schema Registry for independent version discovery and compatibility control."

Why exponential backoff

SAY THIS

"Immediate repeated retries can overload an already failing dependency. Exponential backoff increases the delay after each failure. Bounded retries also guarantee that one message does not retry forever."

Why a Dead Letter Queue

SAY THIS

"A poison message should not block the main stream or disappear silently. The DLQ isolates it, preserves the original business data and adds diagnostic information for later inspection, correction or controlled reprocessing."

Delivery semantics

SAY THIS

"The design is at-least-once oriented, not exactly-once. Manual offset commits reduce message-loss risk because the source offset is recorded only after success or DLQ delivery. However, a crash after processing but before the commit can cause Kafka to replay the event, so duplicate processing remains possible."

8. Honest limitations and production improvements

Current demonstration	Production improvement
One Kafka broker	Use multiple brokers, replicated partitions and appropriate minimum in-sync replicas.
Average stored in consumer memory	Use Kafka Streams, Flink, ksqldb or an external durable state store.
Retry sleeps inside the consumer	Use delayed retry topics so one failed record does not pause its partition.
Schemas deployed with applications	Add a Schema Registry with compatibility policies.
No authentication or encryption	Add TLS, SASL, access-control lists and secret management.
At-least-once behavior	Use transactional processing or idempotent state updates where exact outcomes are required.
No production observability	Add metrics, centralized logs, lag monitoring and alerts.

SAY THIS

"The running average is local and resets when the consumer restarts. Multiple consumers would also calculate separate partial averages. In production I would use durable, partition-aware stream state. The retry loop intentionally simplifies the demonstration; delayed retry topics would improve throughput. The single broker proves functionality but does not provide broker-level fault tolerance."

9. Likely viva questions

Question	Short answer
Why not send orders directly to the consumer?	Kafka decouples services, buffers events, supports replay and permits independent scaling.
Why is <code>orderId</code> the key?	It provides stable partition selection and preserves ordering for events belonging to that order.
Why are failed orders excluded from the average?	Only successfully processed business transactions should change the valid analytic result.
What does <code>acks=all</code> mean here?	The producer waits for acknowledgement from all currently required in-sync replicas; this one-broker demo has only one replica.
Does idempotence make the whole system exactly-once?	No. It reduces producer duplicates, but application processing and state updates can still repeat after a crash.
Why not retry permanent failures?	The same invalid data will fail again, wasting resources and delaying useful work.
What happens if the consumer restarts?	Kafka resumes from committed offsets, but this demo's in-memory running average resets.
Why may logs appear in a different order?	Kafka ordering is per partition. The topic has three partitions, so cross-partition order is not global.
What is KRaft?	Kafka's built-in metadata and controller consensus architecture, replacing the need for ZooKeeper.

10. Final conclusion script

SAY THIS

"In conclusion, the demonstration proves every required feature. Orders are created by a producer, validated against an Avro contract and published as binary records to a partitioned Kafka topic. The consumer performs real-time incremental aggregation. Recoverable failures use bounded exponential retry, while permanent and exhausted failures are retained in an Avro Dead Letter Queue. Manual offset management improves reliability, a separate DLQ consumer provides independent evidence, automated tests pass, and the project is version controlled with Git. Thank you."

11. Recording completion checklist

- ☐ The architecture diagram was clearly visible and explained from left to right.
- ☐ The required Order Avro schema was shown.
- ☐ Producer reliability settings and three demo failure records were shown.
- ☐ `PRODUCED` lines with partitions and offsets were visible.

- ☐ PROCESSED and changing runningAverage lines were visible.
- ☐ The temporary-success case showed two retries and eventual success.
- ☐ The permanent case showed zero retries and DLQ publication.
- ☐ The exhausted case showed three retries and DLQ publication.
- ☐ The independent DLQ consumer showed two DLQ RECEIVED records.
- ☐ The two Kafka topics were listed.
- ☐ The test output showed 8 passed .
- ☐ Git history/status was shown.
- ☐ Limitations and production improvements were stated honestly.
- ☐ The final MP4 was played back and the audio was understandable.

After recording, stop the stack with `docker compose down` . Upload the MP4 to Google Drive, grant the lecturer Viewer access or use "Anyone with the link" if permitted, and test the link in a private browser window.